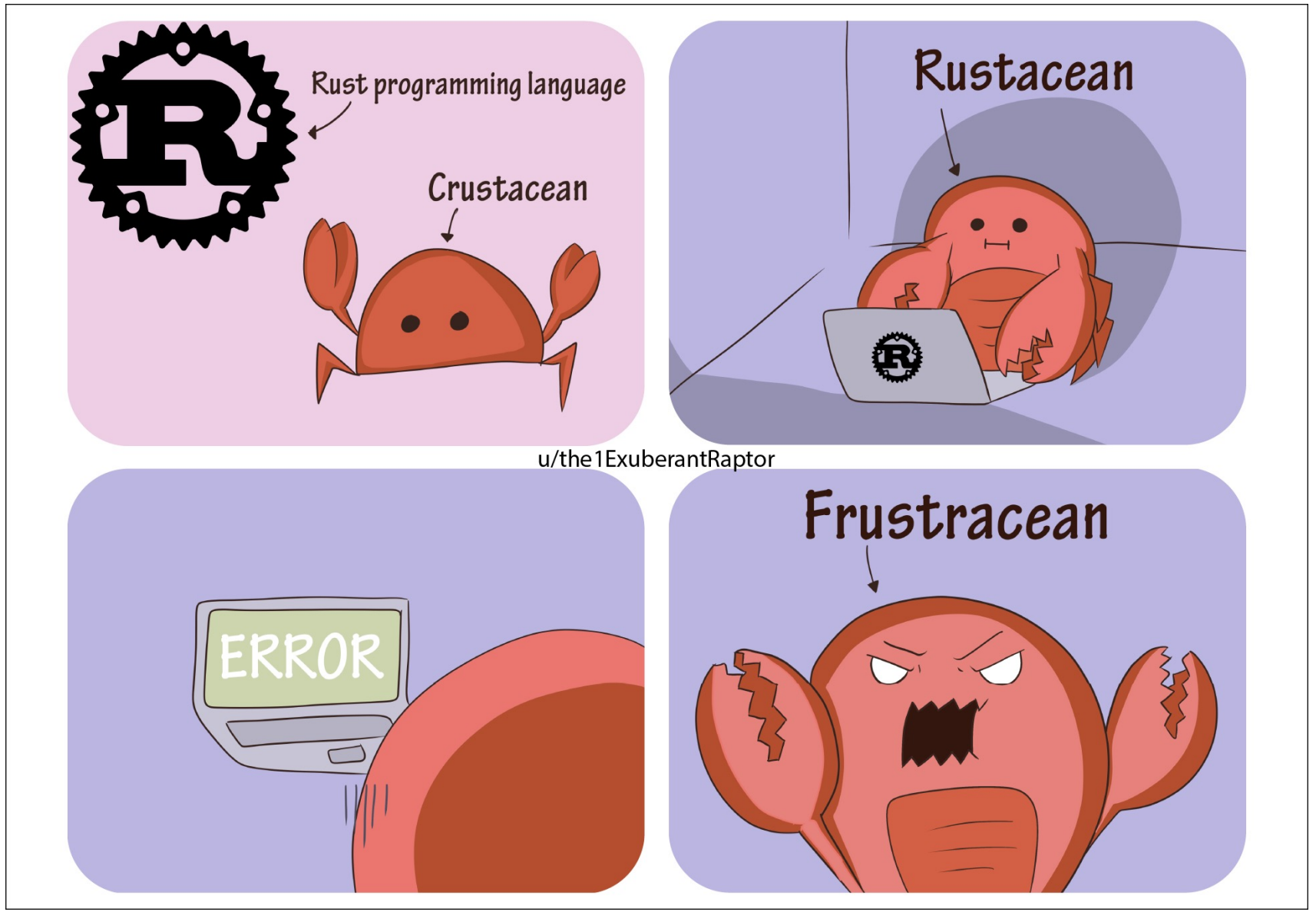
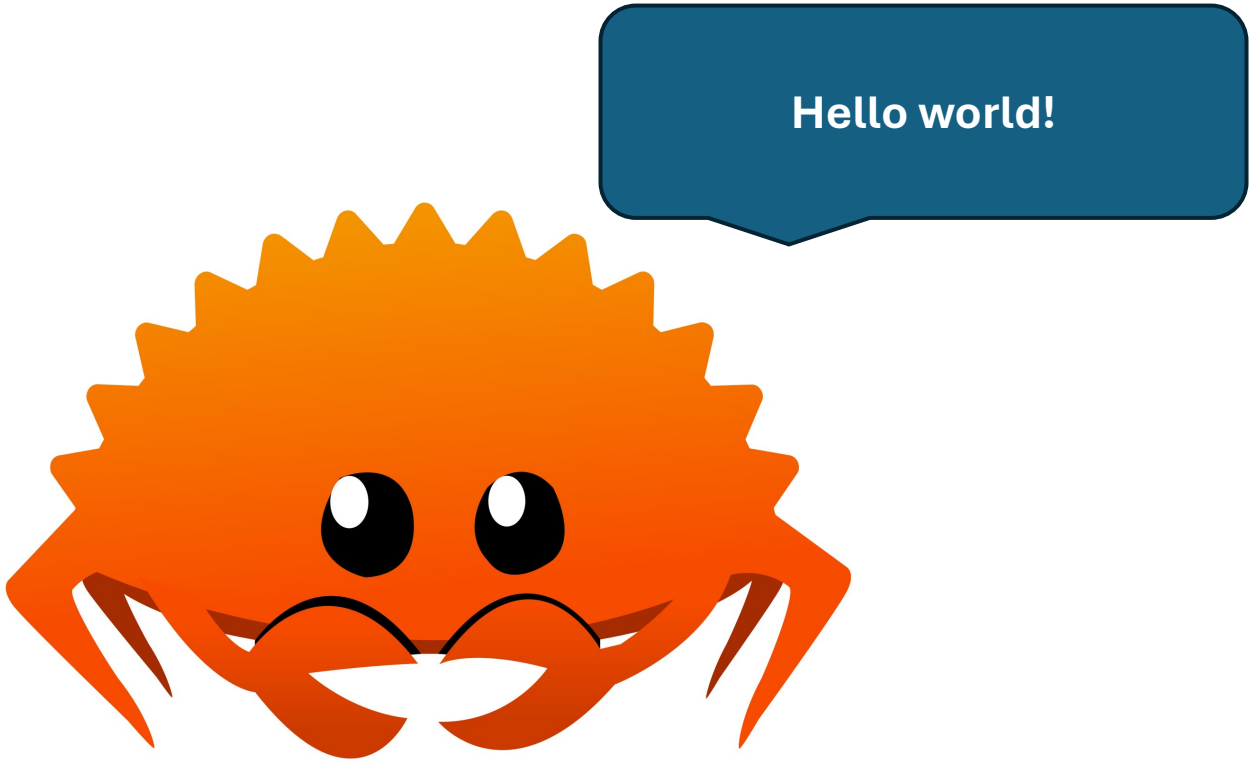

Rust programming language
Crustacean
Rustacean
u/the1ExuberantRaptor
ERROR
Frustracean

What we know up until now



But what does a Rust program look like?

```rust
const N: usize = 9;

fn main() {
    // Sample puzzle, 0 denotes an empty cell.
    let board: [[u8; N]; N] = [
        [5,3,0, 0,7,0, 0,0,0],
        [6,0,0, 1,9,5, 0,0,0],
        [0,9,8, 0,0,0, 0,6,0],

        [8,0,0, 0,6,0, 0,0,3],
        [4,0,0, 8,0,3, 0,0,1],
        [7,0,0, 0,2,0, 0,0,6],

        [0,6,0, 0,0,0, 2,8,0],
        [0,0,0, 4,1,9, 0,0,5],
        [0,0,0, 0,8,0, 0,7,9],
    ];

    println!("Puzzle:");
    print_board(board);

    // Attempt to solve the puzzle.
    let (solved, result) = solve(board);

    if solved {
        println!("\nSolution:");
        print_board(result);
    } else {
        println!("\nNo solution found.");
    }
}
```

Variables and Mutability

- By **default immutable**

```
fn main() {
    let x = 5;
    println!("x = {}", x);
    x = 6;
    println!("x = {}", x);
}
```

Variables and Mutability

- The compiler to the rescue

```
→  l02-test git:(master) ✗ cargo run
   Compiling l02-test v0.1.0 (/home/luca/uni/ws2526/scicomp/l02-presentation/l02-test)
error[E0384]: cannot assign twice to immutable variable `x`
  --> src/main.rs:10:5
   |
 8 |     let x = 5;
   |         - first assignment to `x`
 9 |     println!("x = {}", x);
10 |     x = 6;
   |     ^^^^^ cannot assign twice to immutable variable
   |
help: consider making this binding mutable
   |
 8 |     let mut x = 5;
   |         +++

For more information about this error, try `rustc --explain E0384`.
```

Variables and Mutability

- Can be made **mutable**

```
fn main() {
    let mut x = 5;
    println!("x = {}", x);
    x = 6;
    println!("x = {}", x);
}
```

Variables and Mutability

- Can be made **mutable**

```rust
fn main() {
    let mut x = 5;
    println!("x = {}", x);
    x = 6;
    println!("x = {}", x);
}
```

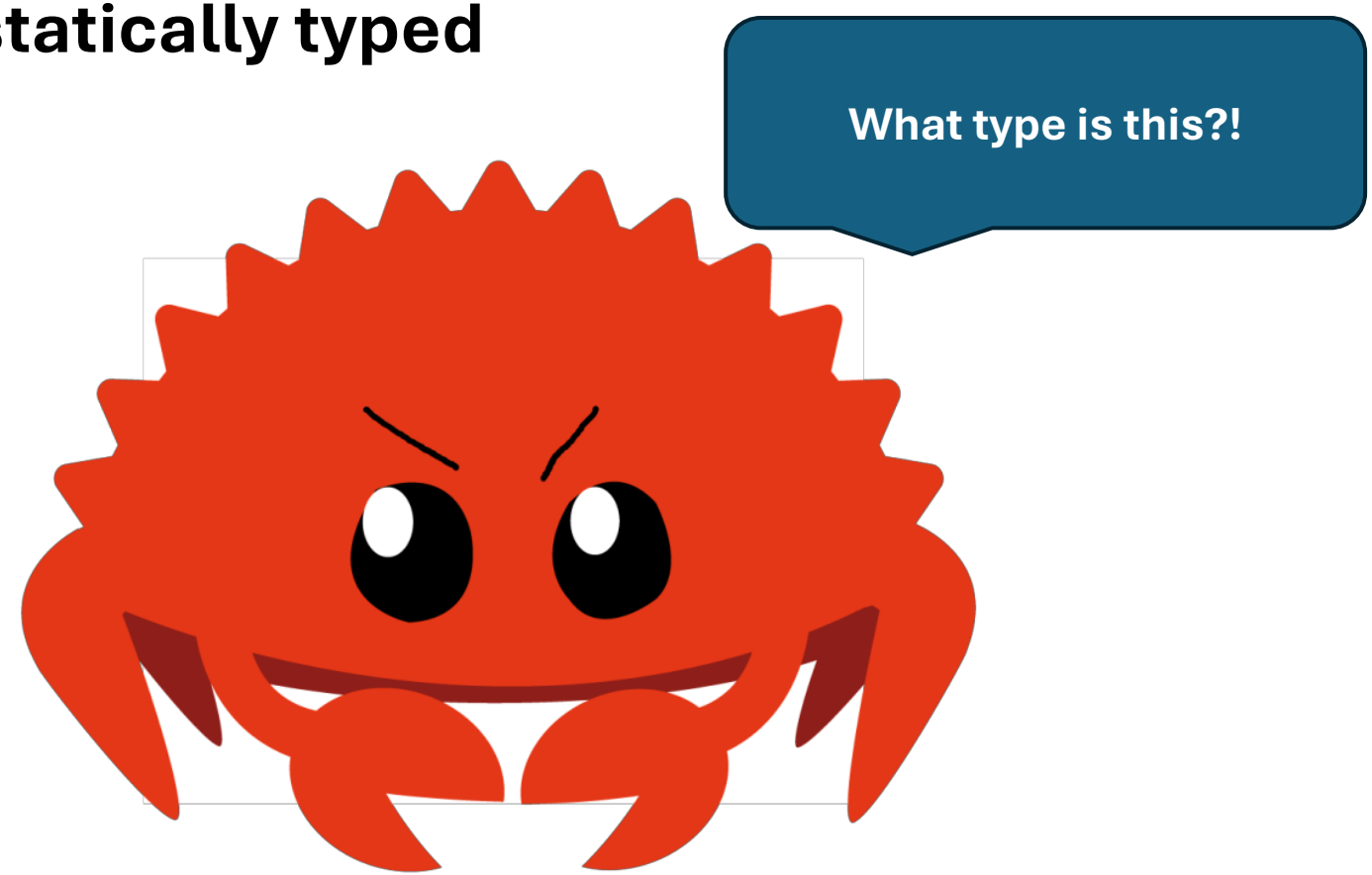
```
→  l02-test git:(master) ✗ cargo run
   Compiling l02-test v0.1.0 (/home/luca/uni/ws2526/scicomp/l02-presentation/l02-test)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
     Running `target/debug/l02-test`
x = 5
x = 6
```

Can we understand this now?

```rust
const N: usize = 9;

fn main() {
    // Sample puzzle, 0 denotes an empty cell.
    let board: [[u8; N]; N] = [
        [5,3,0, 0,7,0, 0,0,0],
        [6,0,0, 1,9,5, 0,0,0],
        [0,9,8, 0,0,0, 0,6,0],

        [8,0,0, 0,6,0, 0,0,3],
        [4,0,0, 8,0,3, 0,0,1],
        [7,0,0, 0,2,0, 0,0,6],

        [0,6,0, 0,0,0, 2,8,0],
        [0,0,0, 4,1,9, 0,0,5],
        [0,0,0, 0,8,0, 0,7,9],
    ];

    println!("Puzzle:");
    print_board(board);

    // Attempt to solve the puzzle.
    let (solved, result) = solve(board);

    if solved {
        println!("\nSolution:");
        print_board(result);
    } else {
        println!("\nNo solution found.");
    }
}
```

Data types

- As always, every value has a certain data type
- BUT! Rust is **statically typed**



Numbers

Length	Signed int	Unsigned int	Float
8-bit	`i8`	`u8`	-
16-bit	`i16`	`u16`	-
32-bit	`i32`	`u32`	`f32`
64-bit	`i64`	`u64`	`f64`
128-bit	`i128`	`u128`	-
architecture dependent	`isize`	`usize`	-

Numbers

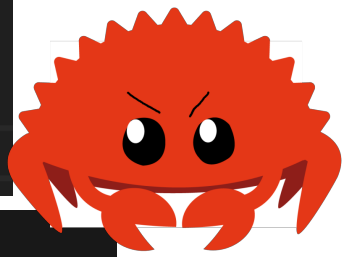
Length	Signed int	Unsigned int	Float
8-bit	`i8`	`u8`	-
16-bit	`i16`	`u16`	-
32-bit	**`i32`**	`u32`	`f32`
64-bit	`i64`	`u64`	**`f64`**
128-bit	`i128`	`u128`	-
architecture dependent	`isize`	`usize`	-

Integer overflow

```rust
fn main() {
    let mut overflow : u8 = 255;
    overflow += 1;
    println!("overflow = {overflow}");
    let mut underflow : u8 = 0;
    underflow -= 1;
    println!("underflow = {underflow}");
}
```

Integer overflow

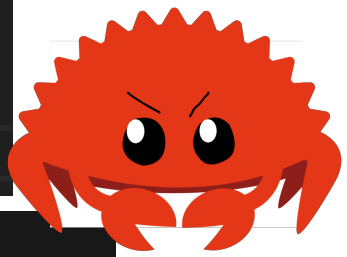
```rust
fn main() {
    let mut overflow : u8 = 255;
    overflow += 1;
    println!("overflow = {overflow}");
    let mut underflow : u8 = 0;
    underflow -= 1;
    println!("underflow = {underflow}");
}
```



```
→  l02-test git:(master) ✗ cargo run
   Compiling l02-test v0.1.0 (/home/luca/uni/ws2526/scicomp/l02-presentation/l02-test)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.15s
     Running `target/debug/l02-test`

thread 'main' panicked at src/main.rs:19:5:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Integer overflow

```rust
fn main() {
    let mut overflow : u8 = 255;
    overflow += 1;
    println!("overflow = {overflow}");
    let mut underflow : u8 = 0;
    underflow -= 1;
    println!("underflow = {underflow}");
}
```



```
→  l02-test git:(master) ✗ cargo run
   Compiling l02-test v0.1.0 (/home/luca/uni/ws2526/scicomp/l02-presentation/l02-test)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.15s
     Running `target/debug/l02-test`

thread 'main' panicked at src/main.rs:19:5:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
→  l02-test git:(master) ✗ cargo run --release
    Finished `release` profile [optimized] target(s) in 0.01s
     Running `target/release/l02-test`
overflow = 0
underflow = 255
```

What can our numbers do?

```rust
fn main() {
    // addition
    let sum = 5 + 10;

    // subtraction
    let difference = 95.5 - 4.3;

    // multiplication
    let product = 4 * 30;

    // division
    let quotient = 56.7 / 32.2;
    let truncated = -5 / 3; // Results in -1

    // remainder
    let remainder = 43 % 5;
}
```

What about characters

- **char** specified with **''**
- Four byte unicode scalar value => can represent far more than simple ASCII

```
fn main() {
    let c = 'z';
    let z: char = 'ℤ'; // with explicit type annotation
    let heart_eyed_cat = '😻';
}
```

Tuples

- Group of **fixed number of values**
- Types may differ

```
let tup = (500, 6.4, "one");
let tup2 : (i32, f64, char) = (500, 6.4, 'a');

let (x, y, z) = tup;
let first_tup = tup.0;
let second_tup = tup.1;
```

Tuples

- Group of **fixed number of values**
- Types may differ

```
let tup = (500, 6.4, "one");
let tup2 : (i32, f64, char) = (500, 6.4, 'a');

let (x, y, z) = tup;
let first_tup = tup.0;
let second_tup = tup.1;
```

Arrays

- Group of **fixed number of values**
- **All entries of the same type**

```
let arr1 = [10, 20, 30, 40, 50];
let arr2 : [i32; 5] = [1, 2, 3, 4, 5];
let arr3 = [3 ; 5] ; // [3, 3, 3, 3, 3]

let first = arr1[0];
let second = arr1[1];
let size = arr1.len();
```

Can we understand this now?

```rust
const N: usize = 9;

fn main() {
    // Sample puzzle, 0 denotes an empty cell.
    let board: [[u8; N]; N] = [
        [5,3,0, 0,7,0, 0,0,0],
        [6,0,0, 1,9,5, 0,0,0],
        [0,9,8, 0,0,0, 0,6,0],

        [8,0,0, 0,6,0, 0,0,3],
        [4,0,0, 8,0,3, 0,0,1],
        [7,0,0, 0,2,0, 0,0,6],

        [0,6,0, 0,0,0, 2,8,0],
        [0,0,0, 4,1,9, 0,0,5],
        [0,0,0, 0,8,0, 0,7,9],
    ];

    println!("Puzzle:");
    print_board(board);

    // Attempt to solve the puzzle.
    let (solved, result) = solve(board);

    if solved {
        println!("\nSolution:");
        print_board(result);
    } else {
        println!("\nNo solution found.");
    }
}
```

Functions

- Just like main, we define other functions
- You **must** define **type annotations!**

```
fn main() {
    let num = 5;
    println!("{}", add(num, 10));
}
fn add(x: i32, y : i64) -> i32 {
    x + y as i32
}
```

Can we understand this now?

```rust
fn solve(board: [[u8; N]; N]) -> (bool, [[u8; N]; N]) {
    // pick the next empty cell
    let (found, row, col) = find_empty(board);

    // no empty cell - solved
    if !found {
        return (true, board);
    }

    // try values 1 through 9 in the chosen cell
    let mut val: u8 = 1;
    while val <= 9 {
        if is_valid(board, row, col, val) {
            // create a new board state with the value placed
            let mut new_board = board;
            new_board[row][col] = val;

            // Recurse on the updated board
            let (solved, result_board) = solve(new_board);
            if solved {
                // Solved!
                return (true, result_board);
            }
            // Otherwise, try next value.
        }
        val += 1;
    }

    // Tried all values and none worked
    (false, board)
}
```

```rust
const N: usize = 9;

fn main() {
    // Sample puzzle, 0 denotes an empty cell.
    let board: [[u8; N]; N] = [
        [5,3,0, 0,7,0, 0,0,0],
        [6,0,0, 1,9,5, 0,0,0],
        [0,9,8, 0,0,0, 0,6,0],

        [8,0,0, 0,6,0, 0,0,3],
        [4,0,0, 8,0,3, 0,0,1],
        [7,0,0, 0,2,0, 0,0,6],

        [0,6,0, 0,0,0, 2,8,0],
        [0,0,0, 4,1,9, 0,0,5],
        [0,0,0, 0,8,0, 0,7,9],
    ];

    println!("Puzzle:");
    print_board(board);

    // Attempt to solve the puzzle.
    let (solved, result) = solve(board);

    if solved {
        println!("\nSolution:");
        print_board(result);
    } else {
        println!("\nNo solution found.");
    }
}
```

If - else if - else

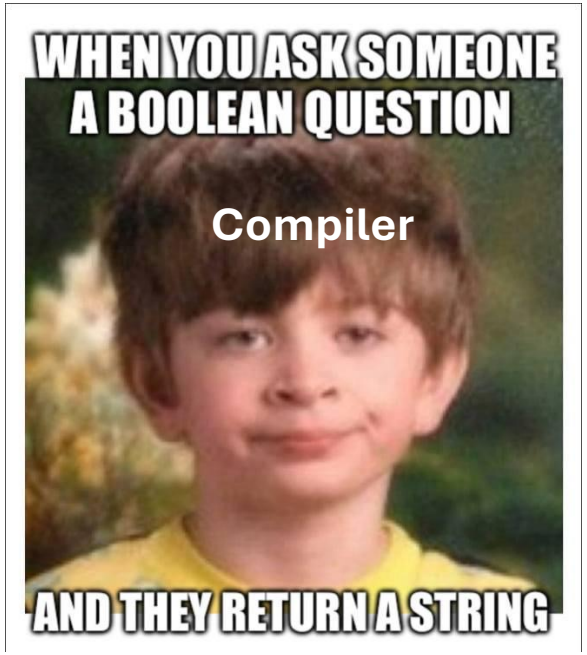
- Need to evaluate a **bool value**
- No automatic conversion to bool, be explicit!

```
fn main() {
    let number = 6;

    if number % 4 == 0 {
        println!("number is divisible by 4");
    } else if number % 3 == 0 {
        println!("number is divisible by 3");
    } else if number % 2 == 0 {
        println!("number is divisible by 2");
    } else {
        println!("number is not divisible by 4, 3, or 2");
    }
}
```

If - else if - else

- Need to evaluate a **bool value**
- No automatic conversion to bool, be explicit!

```rust
fn main() {
    let number = 6;

    if number % 4 == 0 {
        println!("number is divisible by 4");
    } else if number % 3 == 0 {
        println!("number is divisible by 3");
    } else if number % 2 == 0 {
        println!("number is divisible by 2");
    } else {
        println!("number is not divisible by 4, 3, or 2");
    }
}
```



While and for

- Work as usual
- We can also use ***break*** and ***continue***

```
fn main() {
    let mut number = 3;

    while number != 0 {
        println!("{number}!");

        number -= 1;
    }

    println!("LIFTOFF!!!");
}
```

While and for

- Work as usual
- We can also use ***break*** and ***continue***

```
fn main() {
    let mut number = 3;

    while number != 0 {
        println!("{number}!");

        number -= 1;
    }

    println!("LIFTOFF!!!");
}
```

```
fn main() {
    for number in (1..4).rev() {
        println!("{number}!");
    }
    println!("LIFTOFF!!!");
}
```

While and for

- Work as usual
- We can also use ***break*** and ***continue***

```
fn main() {
    let mut number = 3;

    while number != 0 {
        println!("{number}!");

        number -= 1;
    }

    println!("LIFTOFF!!!");
}
```

```
fn main() {
    for number in (1..4).rev() {
        println!("{number}!");
    }
    println!("LIFTOFF!!!");
}
```

Can we understand this now?

```rust
fn solve(board: [[u8; N]; N]) -> (bool, [[u8; N]; N]) {
    // pick the next empty cell
    let (found, row, col) = find_empty(board);

    // no empty cell - solved
    if !found {
        return (true, board);
    }

    // try values 1 through 9 in the chosen cell
    let mut val: u8 = 1;
    while val <= 9 {
        if is_valid(board, row, col, val) {
            // create a new board state with the value placed
            let mut new_board = board;
            new_board[row][col] = val;

            // Recurse on the updated board
            let (solved, result_board) = solve(new_board);
            if solved {
                // Solved!
                return (true, result_board);
            }
            // Otherwise, try next value.
        }
        val += 1;
    }

    // Tried all values and none worked
    (false, board)
}
```

```rust
const N: usize = 9;

fn main() {
    // Sample puzzle, 0 denotes an empty cell.
    let board: [[u8; N]; N] = [
        [5,3,0, 0,7,0, 0,0,0],
        [6,0,0, 1,9,5, 0,0,0],
        [0,9,8, 0,0,0, 0,6,0],

        [8,0,0, 0,6,0, 0,0,3],
        [4,0,0, 8,0,3, 0,0,1],
        [7,0,0, 0,2,0, 0,0,6],

        [0,6,0, 0,0,0, 2,8,0],
        [0,0,0, 4,1,9, 0,0,5],
        [0,0,0, 0,8,0, 0,7,9],
    ];

    println!("Puzzle:");
    print_board(board);

    // Attempt to solve the puzzle.
    let (solved, result) = solve(board);

    if solved {
        println!("\nSolution:");
        print_board(result);
    } else {
        println!("\nNo solution found.");
    }
}
```

The TLDR

- In general Rust does not reinvent the wheel – apply what you **know**
- **Listen** to the Compiler
- **Think** about what a variable should do and what traits it needs when creating it

The TLDR

- In general Rust looks and works like other languages – apply what you **know**
- **Listen** to the Compiler
- **Think** about what a variable should do and what traits it needs when creating it

Some useful stuff to look up

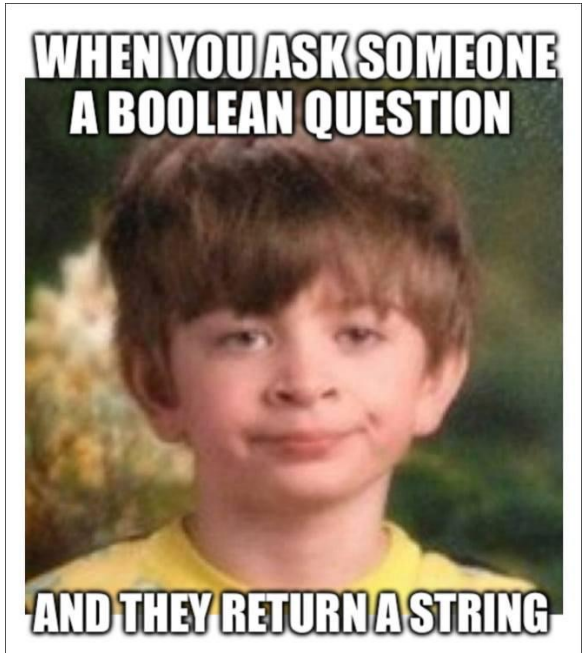
- Importing and using packages, e.g. ***use rand::Rng*** (Chapter 2)
  - Try in terminal: ***cargo doc –open***
- Shadowing (as seen in the exercise) (Chapter 3.1)
- Expression vs. Statement (Chapter 3.3)
- Using if in a let statement (Chapter 3.5)
- General overview over different concepts (Chapter 2)

We forgot one type - Bool

- One byte
- Work as usual and mainly used in control flow

```rust
fn main() {
    let mut yes : bool = true;
    yes = "But, ...";
    println!("{yes}");
}
```

```
   Compiling l02-test v0.1.0 (/home/luca/uni/ws2526/scicomp/l02-presentation/l02-test)
error[E0308]: mismatched types
 --> src/main.rs:6:11
  |
4 |     let mut yes : bool = true;
  |                  ---- expected due to this type
5 |
6 |     yes = "But, ...";
  |           ^^^^^^^^^^ expected `bool`, found `&str`

For more information about this error, try `rustc --explain E0308`.
error: could not compile `l02-test` (bin "l02-test") due to 1 previous error
```



If is an expression

- Need to evaluate to a **bool value**
- No automatic conversion to bool, be explicit!
- All outcomes must be of the same type

```
fn main() {
    let condition = true;
    let number = if condition { 5 } else { 6 };

    println!("The value of number is: {number}");
}
```

```
The value of number is: 5
```

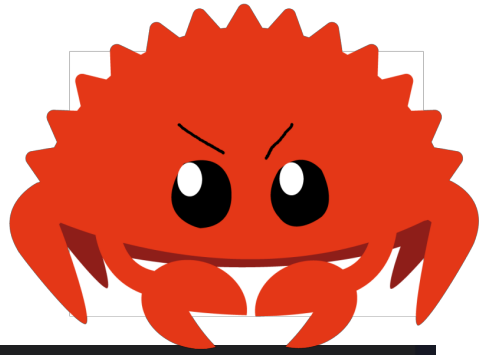
If is an expression

- Need to evaluate to a **bool value**
- No automatic conversion to bool, be explicit!
- All outcomes must be of the same type

```
fn main() {
    let condition = true;

    let number = if condition { 5 } else { "six" };

    println!("The value of number is: {number}");
}
```

If is an expression

- Need to evaluate to a **bool value**
- No automatic conversion to bool, be explicit!
- All outcomes must be of the same type



```
$ cargo run
   Compiling branches v0.1.0 (file:///projects/branches)
error[E0308]: `if` and `else` have incompatible types
 --> src/main.rs:4:44
  |
4 |     let number = if condition { 5 } else { "six" };
  |                                 -          ^^^^^ expected integer, found `&s
  |                                 |
  |                                 expected because of this

For more information about this error, try `rustc --explain E0308`.
error: could not compile `branches` (bin "branches") due to 1 previous error
```

loop

- loop is ***while true{}***
- While is used as always
- As usual we have ***break*** and ***continue***

```
fn main() {
    // A stupid way to check for overflow
    loop {
        let mut overflow : u8 = 199;
        overflow = overflow + 10;
        println!("count = {overflow}");
        if overflow < 10 {
            break;
        }
    }
    println!("Overflow happened!");
}
```

Can we understand this now?

```
fn solve(board: [[u8; N]; N]) -> (bool, [[u8; N]; N]) {
    let (found, row, col) = find_empty(board);

    if !found {
        return (true, board);
    }

    let mut val: u8 = 1;
    while val <= 9 {
        if is_valid(board, row, col, val) {
            let mut new_board = board;
            new_board[row][col] = val;

            let (solved, result_board) = solve(new_board);
            if solved {
                return (true, result_board);
            }
        }
        val += 1;
    }

    (false, board)
}
```

```
fn is_valid(board: [[u8; N]; N], row: usize, col: usize, val: u8) -> bool {
    let mut i = 0;
    while i < N {
        if board[row][i] == val { return false; }
        if board[i][col] == val { return false; }
        i += 1;
    }

    let br = (row / 3) * 3;
    let bc = (col / 3) * 3;
    for rr in br..br + 3 {
        for cc in bc..bc + 3 {
            if board[rr][cc] == val { return false; }
        }
    }
    true
}
```